

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)**

Институт информационных технологий, математики и механики

**Кафедра: высокопроизводительных вычислений и системного
программирования**

Направление подготовки: «Фундаментальная информатика и
информационные технологии»

Профиль подготовки: «Инженерия программного обеспечения»

ОТЧЕТ

по учебной практике

на тему:

**«Разработка стекового процессора на базе программируемой
логической интегральной схемы с использованием языка
описания аппаратуры SystemVerilog»**

Выполнил: студент
группы 3823Б1ФИЗ

_____ Гутянский А.С.

Научный руководитель:
доцент кафедры ВВиСП, к.т.н.,
_____ Козин Е.А.

Оглавление

Введение	3
1. Постановка задачи	4
2. Реализация процессора	5
1.1. Аппаратная платформа.....	5
1.2. Выбор инструментов для разработки	5
1.3. Стековая и регистровая модели вычислений	6
1.4. Структура памяти	7
1.5. Подсистема ввода/вывода	8
1.6. Структура вычислительной системы	9
1.7. Внутренние регистры процессора.....	9
1.8. Структура инструкций	10
1.9. Цикл исполнения инструкции	13
1.10. Описание функциональных блоков.....	15
3. Реализация чтения и записи программ через UART.....	18
3.1. Протокол UART	18
3.2. Использование протокола	19
4. Реализация компилятора.....	21
4.1. Описание языка программирования	21
4.2. Работа с памятью	23
4.3. Генерация кода.....	23
4.4. Примеры программ.....	26
Заключение.....	28
Список литературы.....	29
Приложение А. Исходный код	30

Введение

Современные вычислительные системы характеризуются высокой степенью абстракции как на аппаратном, так и на программном уровне. Большинство разработчиков взаимодействуют с процессорами через многоуровневые программные интерфейсы, не имея прямого доступа к архитектурным решениям, лежащим в основе исполнения программ. В результате фундаментальные принципы работы вычислительных машин – организация памяти, исполнение инструкций, взаимодействие аппаратуры и транслятора – остаются скрытыми и недостаточно наглядными.

В то же время изучение архитектуры процессоров и принципов их проектирования остаётся актуальной задачей в области подготовки специалистов по вычислительной технике и системному программированию. Особенно важным является практический опыт разработки вычислительной системы с нуля, охватывающий весь цикл – от формализации архитектурных решений до реализации аппаратуры и программного обеспечения. Такой подход позволяет лучше понять компромиссы между сложностью аппаратной реализации, производительностью и удобством программирования.

Программируемые логические интегральные схемы (ПЛИС) предоставляют удобную платформу для экспериментального проектирования процессоров. Они позволяют реализовывать и модифицировать архитектуры без изготовления специализированных микросхем, что делает ПЛИС особенно востребованными в образовательных и исследовательских проектах. Использование языков описания аппаратуры, таких как SystemVerilog, обеспечивает формальное и воспроизводимое описание архитектуры, пригодное как для моделирования, так и для запуска на реальном оборудовании.

Отдельный интерес представляет стековая модель вычислений. Её ключевым преимуществом является простота аппаратной реализации и упрощённый процесс трансляции программ, что делает стековую архитектуру удобным объектом для учебного проектирования и экспериментов с системами команд и компиляторами.

1. Постановка задачи

Целью данной работы является разработка учебного процессора со стековой архитектурой, реализованного на программируемой логической интегральной схеме, а также создание минимального программного инструментария, обеспечивающего выполнение тестовых программ на данной архитектуре.

Для достижения поставленной цели в рамках работы необходимо решить следующие задачи:

1. Разработать архитектуру процессора, включая систему команд, модель памяти и организацию вычислений на основе стековой модели.
2. Реализовать аппаратную часть процессора с использованием языка описания аппаратуры SystemVerilog, обеспечив корректное выполнение всех инструкций системы команд.
3. Спроектировать механизм взаимодействия процессора с памятью программ и памятью данных, включая поддержку вывода через отображаемые в память регистры.
4. Разработать средство загрузки программ в память через внешний интерфейс.
5. Создать простой компилятор для разработанной архитектуры, преобразующий программы на высокоуровневом языке в машинный код процессора.
6. Обеспечить поддержку базовых конструкций высокоуровневого языка, включая переменные, арифметические выражения, управляющие конструкции и подпрограммы.

В рамках работы не ставится задача оптимизации производительности, реализации сложных механизмов защиты памяти или поддержки расширенных типов данных. Основное внимание уделяется наглядности архитектурных решений, корректности исполнения инструкций и согласованности аппаратной и программной частей системы.

2. Реализация процессора

1.1. Аппаратная платформа

Для реализации процессора использована отладочная плата Terasic DE1-SoC, основанная на ПЛИС Intel (Altera) Cyclone V. Выбор обусловлен несколькими факторами, важными для учебных и исследовательских проектов:

- Наличие значительного объёма внутреннего двухпортового ОЗУ. Это позволило разместить память инструкций и данных внутри ПЛИС, обеспечив высокую скорость доступа и упростив подключение UART-загрузчика.
- Высокая плотность логики при низком энергопотреблении. Cyclone V даёт достаточно ресурсов (ALM-элементов, регистров и PLL), чтобы реализовать все необходимые модули.
- Удобная интеграция с инструментами Intel. Плата официально поддерживается Quartus Prime, ModelSim, фирменными IP-ядрами и библиотеками.
- Доступность и распространенность DE1-SoC в учебных курсах. Большое количество примеров и материалов упрощает изучение и помогает в решении возникающих проблем.

Эта платформа оптимально подходит для построения учебных процессорных архитектур: доступна, хорошо документирована и содержит нужный набор периферии.

1.2. Выбор инструментов для разработки

Подбор инструментария для разработки процессора был обусловлен используемой аппаратной платформой – отладочной платой Terasic DE1-SoC с ПЛИС Altera Cyclone V. Это определило «стек» ПО для всего цикла проектирования.

Intel Quartus Prime Lite Edition выбран как основной инструмент синтеза и размещения логики на ПЛИС. Без Quartus невозможно перевести RTL-описание на SystemVerilog в конфигурирующий файл для ПЛИС фирмы Altera. Помимо генерации битового потока, среда включает широкий набор важных при разработке инструментов: конфигуратор IP-ядер, редактор содержимого встроенной памяти микросхемы, редактор назначения выводов и другие.

Для функциональной верификации до загрузки на железо использовался ModelSim (Intel Edition). Основным критерием для выбора послужила простота интеграции с Quartus. Для запуска симуляции достаточно выбрать testbench для отладки и нажать кнопку запуска. ModelSim автоматически компилирует код, загружает необходимые библиотеки, в том числе библиотеки поддержки специфичных для Altera блоков, проводит симуляцию и настраивает необходимые виджеты для визуализации собранных данных. Симуляция позволяет выявить ошибки, допущенные при проектировании и написании кода без запуска синтеза, экономя время на итерациях с ПЛИС.

Ввиду того, что встроенный в Quartus редактор кода имеет достаточно скудный функционал, для написания кода была выбрана среда Visual Studio Code с установленным расширением для поддержки SystemVerilog от норвежского разработчика Eirik Prestegårdshus. Расширение добавляет подсветку синтаксиса языка, автоматическое дополнение кода и статический анализ для обнаружения ошибок. Среди недостатков стоит

отметить низкую производительность при работе с проектами большого размера (сотни тысяч объектов, описанных в коде).

1.3. Стековая и регистровая модели вычислений

Выбор модели вычислений, несомненно, является приоритетным вопросом в самом начале проектирования вычислительной системы. Сделанный выбор заранее определяет следующие шаги разработки: проектирование микроархитектуры процессора, набора поддерживаемых команд, транслятора для языка. Для обоснования выбора приведем сравнительный анализ стековой и регистровой моделей вычислений по небольшому набору ключевых параметров:

Таблица 1. Сравнение стековой и регистровой архитектур

Критерий	Стековая модель	Регистровая модель
Принцип работы	Операции выполняются над операндами, лежащими на вершине стека; результат помещается обратно на стек.	Данные хранятся в явно адресуемых регистрах; инструкции явно указывают регистры-источники и регистр-приемник.
Управление ресурсами	Не требуется управление регистрами.	Требуется решение задачи распределения регистров.
Сложность компиляции	Не нужен анализ жизненного цикла переменных и распределение регистров.	Требуется алгоритмы распределения регистров и оптимизации использования ресурсов.
Плотность кода	Код компактнее за счет отсутствия в нем адресов операндов и места сохранения результата.	Инструкции должны явно содержать номера регистров, с которыми они работают.
Производительность	Частые операции со стеком и невозможность повторного использования операндов дают большую нагрузку на подсистему памяти.	Работа с внутренними регистрами происходит намного эффективнее, чем с внешней памятью; выбор места сохранения результата позволяет повторно использовать аргументы, когда это необходимо.
Гибкость архитектуры	Произвольный доступ к операндам (в том числе переупорядочивание) требует введения на уровне системы команд специализированных инструкций.	Есть произвольный доступ к регистрам в любом порядке.

Механизм трансляции	Преобразование выражения в обратную польскую запись.	Номер регистра должен быть явно указан в инструкции, что требует дополнительной работы по распределению ресурсов.
Типичные области применения	Виртуальные машины (JVM, PVM и другие), где важна простота трансляции и кроссплатформенность.	Современные высокопроизводительные процессоры (x86, ARM, RISC-V), некоторые производительные машины (например, Dalvik VM), где критична производительность.

Анализ показывает, что регистровая модель предоставляет больше возможностей для низкоуровневой оптимизации и обеспечивает лучшую производительность, но в то же время задает серьезные требования к реализации транслятора.

Приоритет в работе отдан проектированию вычислительной системы, а не разработке для нее сложного транслятора. Поэтому, несмотря на потенциально более низкую производительность, для реализации выбрана стековая модель.

1.4. Структура памяти

В современной вычислительной технике существуют две фундаментальные модели организации памяти:

Архитектура фон Неймана (принстонская архитектура):

- единое пространство памяти для команд и данных;
- общая шина для передачи команд и данных;
- простота реализации и универсальность.

Гарвардская архитектура:

- отдельные памяти для команд и данных, каждая со своей шиной;
- параллельный доступ к командам и данным;
- повышенный контроль над исполнением кода.

Таблица 2. Сравнение Гарвардской архитектуры и архитектуры фон Неймана

Критерий	Архитектура фон Неймана	Гарвардская архитектура
Организация памяти	Единое пространство для команд и данных	Раздельная память для команд и данных
Шины данных	Общая шина для команд и данных	Отдельные шины для команд и данных
Параллелизм	Последовательный доступ	Параллельный доступ к командам и данным

Гибкость	Высокая (код может быть модифицирован при исполнении)	Ограниченная (присутствует жесткое разделение областей)
Сложность реализации	Ниже (меньше компонентов)	Выше (двойная система памяти и шин)
Производительность	Ограничивается конфликтами на общей шине	Выше за счет параллельного доступа
Типичное применение	Универсальные компьютеры, серверы	Микроконтроллеры, сигнальные процессоры, специализированные системы

Для разрабатываемого процессора была выбрана Гарвардская архитектура. Отделение места хранения программы от места хранения переменных позволяет избежать ошибок, связанных с непреднамеренной перезаписью исполняемого кода. Однако, целенаправленное изменение кода во время исполнения также становится невозможным. Ситуации, когда такое поведение может быть оправдано рассматриваться в рамках данной работы не будут.

В качестве ширины машинного слова выбрано значение 4 байта. Работа с памятью может быть выполнена только пословно, невыровненный доступ невозможен. Все операции, производимые процессором, принимают 4-байтовые аргументы и выдают 4-байтовый результат, что даёт достаточный для большинства операций диапазон значений.

И память программ, и память данных содержат по 4096 четырехбайтовых ячеек. Указанный объем достаточен для выполнения большинства тестовых программ и прототипирования алгоритмов.

Внутренняя память в ПЛИС состоит из жестко заданных блоков. Объединение нескольких блоков в устройство, способное сохранить больше данных, потребует создания дополнительных логических элементов, увеличивающих время прохождения сигналов, а значит уменьшающих максимальную частоту работы процессора.

1.5. Подсистема ввода/вывода

В разрабатываемом процессоре реализована подсистема ввода-вывода по принципу Memory-Mapped I/O (MMIO) – периферийные устройства отображаются в общем адресном пространстве памяти данных, а операции ввода-вывода выполняются стандартными командами чтения/записи в память.

Основным элементом вывода является 32-битный выходной регистр, размещённый по фиксированному адресу:

$$\text{Адрес} = 1 \ll 30 = 2^{30} = 0x40000000$$

Младшие 16 бит регистра транслируются на четыре семисегментных индикатора, которые отображают значение в шестнадцатеричном формате:

- первый индикатор – биты 0–3;
- второй индикатор – биты 4–7;
- третий индикатор – биты 8–11;
- четвертый индикатор – биты 12–16.

В текущей реализации процессора отсутствует явная возможность ввода данных. Указание обрабатываемых данных происходит путем предварительного размещения констант в памяти программ. В процессе выполнения программы константы кладутся на стек и могут быть либо сразу использованы для вычислений, либо помещены в память данных для использования в будущем.

Пример использования: программа вычисляет некоторое значение (например, результат арифметической операции). Результат записывается в выходной регистр по адресу 0x40000000. Индикаторы автоматически отображают младшие 16 бит значения в шестнадцатеричном виде.

Для нового расчёта программа загружается с иными константами в памяти программ, и процесс повторяется.

Таким образом, обеспечивается базовый ввод-вывод, достаточный для отладки и демонстрации работы процессора.

1.6. Структура вычислительной системы

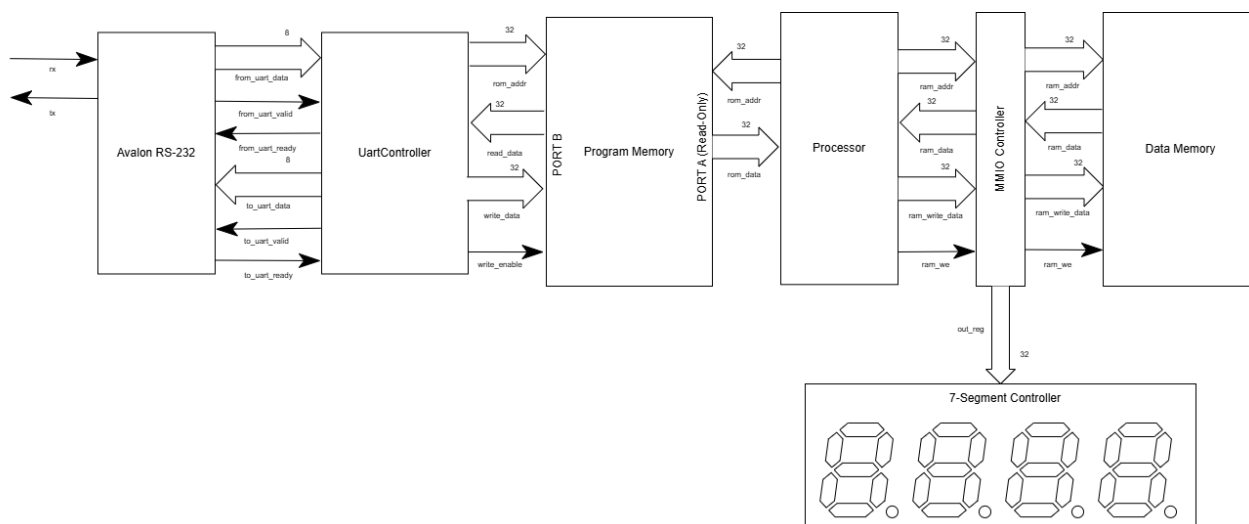


Рисунок 1. Структура вычислительной системы

Как было определено ранее, в архитектуре процессора используются два типа памяти: память программ и память данных. В связи с этим процессор оснащён двумя независимыми комплектами шин для взаимодействия с каждым из этих типов памяти.

Доступ к памяти данных осуществляется через контроллер ММЮ, который перенаправляет все операции чтения и записи по адресу 0x40000000 на выделенный регистр вывода.

Память программ реализована в виде двухпортового ОЗУ. При этом порт А, используемый процессором, предназначен исключительно для чтения инструкций и не поддерживает запись данных. Порт В используется для загрузки программ по протоколу UART. Подробное описание данного механизма будет приведено в одном из следующих разделов.

Далее рассматривается внутреннее устройство разработанного процессора.

1.7. Внутренние регистры процессора

Несмотря на то, что процессор реализует стековую модель вычислений, он все же содержит несколько внутренних 32-битных регистров для сохранения состояния процесса

вычислений. При сбросе значения всех регистров устанавливаются в 0, что является корректным для начала исполнения, если точка входа программы расположена по нулевому адресу.

Адрес следующей инструкции, которая должна быть обработана сохраняется регистром PC. Как только инструкция была декодирована, происходит увеличение значения регистра на единицу. Значение регистра перезаписывается при выполнении инструкции перехода.

При выполнении инструкции перехода, значение регистра PC перед изменением копируется в регистр PPC. Этот регистр может быть использован при реализации корректного возврата из подпрограмм.

Стек вычислений начинается в нулевой ячейке памяти данных и продолжается в направлении увеличения адресов. Вершиной стека называется первая незанятая ячейка. Её адрес сохраняется регистром SP. При записи значения на стек, значение регистра увеличивается на 1, а при чтении уменьшается на 1.

Для ускорения вычисления адресов локальных переменных функций используется регистр FP – указатель фрейма функции. Он содержит адрес, относительно которого по заданному в инструкции смещению (положительному или отрицательному) может быть произведено чтение или запись памяти данных.

Память данных имеет единственный набор шин для взаимодействия, поэтому чтение операндов инструкции должно быть выполнено последовательно. Для сохранения прочитанных значений используются регистры A и B.

На самом деле не все регистры процессора являются 32-битными. Прочитанная из памяти инструкция декодируется и записывается в набор управляющих регистров, имеющих различную ширину в зависимости от их назначения. Регистры содержат управляющие коды для функциональных блоков.

1.8. Структура инструкций

Все инструкции имеют фиксированную длину – 32 бита. В зависимости от значения старшего бита они делятся на два типа.

Если старший бит равен нулю, происходит загрузка константы на стек:

$$\text{Инструкция} = 0 \mid \text{Константа (31 бит)}$$

Знак ‘|’ в выражении означает конкатенацию битовых строк. Младшие биты команды задают беззнаковую 31-битную константу (imm). Инструкцию в целом можно распознавать как неотрицательное 32-битное число, где для указания знака используется дополнительный код.

Если старший бит равен единице, то в инструкции указывается режим адресации, операция АЛУ и 22-битная знаковая константа:

$$\text{Инструкция} = 1 \mid \text{Константа (22 бита)} \mid \text{Операция (5 бит)} \mid \text{Режим адресации (4 бита)}$$

Константа распознается как число, записанное в дополнительном коде. Перед использованием происходит расширение до 32-битного значения с учетом знака.

Таблица 3. Режимы адресации

Код (4 бита)	Режим адресации	Описание
0000	stack, stack => stack	Взять значения со стека, применить операцию, положить результат на стек.
0001	stack, imm => stack	Взять в качестве левого операнда значение со стека, взять в качестве правого операнда константу, положить результат на стек.
0010	[B], imm => stack	Положить на стек значение, хранящееся в памяти по адресу, задаваемому вторым операндом В. Операнд В задается константой.
0011	stack, imm => [B]	Сохранить в памяти значение со стека по адресу, задаваемому операндом В. Операнд В задается константой.
0100	[B], stack => stack	Положить на стек значение, хранящееся в памяти по адресу, задаваемому вторым операндом В. Операнд В загружается со стека.
0101	stack, stack => [B]	Сохранить в памяти значение со стека по адресу, задаваемому операндом В. Операнд В загружается со стека.
0110	[B], FP + imm => stack	Положить на стек значение, хранящееся в памяти по адресу, задаваемому константой, указателем фрейма функции и некоторой выполняемой над ними операцией.
0111	stack, FP + imm => [B]	Взятое со стека значение записать в память по адресу, задаваемому константой, указателем фрейма функции и некоторой выполняемой над ними операцией.

1000	FP, imm => stack	Взять в качестве операндов указатель фрейма функции и константу, выполнить операцию, положить результат на стек.
1001	stack, imm => FP	Взять в качестве операндов значение со стека и константу, выполнить операцию, записать указатель фрейма функции.
1010	FP, imm => FP	Взять в качестве операндов указатель фрейма функции и константу, выполнить операцию, записать указатель фрейма функции.
1011	imm, stack => PC	Взять в качестве операндов константу и значение со стека. Перейти по адресу, содержащемуся в операнде В.
1100	stack, stack => A == 0 ? PC : nop	Взять со стека значение двух операндов. Если операнд А равен нулю, выполнить переход по адресу, содержащемуся в операнде В, также взятому со стека.
1101	stack, stack => A != 0 ? PC : nop	Взять со стека значение двух операндов. Если операнд А равен нулю, выполнить переход по адресу, содержащемуся в операнде В, также взятому со стека.
1110	PPC, imm => stack	Взять в качестве операндов значение rrc (значение pc до выполнения перехода) и константу, выполнить операцию. Положить на стек.
1111	imm, imm => nop	Операция не содержит значимого результата (пропуск цикла).

Допустимые коды операций АЛУ (все операции происходят над 32-битными знаковыми числами, записанными в дополнительном коде):

Таблица 4. Операции, поддерживаемые АЛУ

Код (5 бит)	Операция	Описание
00000	0	Результат равен нулю
00001	-A	Взять A с противоположным знаком
00010	A + B	Сумма A и B
00011	A - B	Разность A и B
00100	A * B	Произведение A и B
00101	A / B	Целочисленное деление A на B
00110	A % B	Остаток от целочисленного деления A на B
00111	A & B	Побитовое И от A и B
01000	A B	Побитовое ИЛИ от A и B
01001	~A	Побитовое отрицание A
01010	A ^ B	Побитовое исключающее ИЛИ от A и B
01011	A	Результат равен A
01100	B	Результат равен B
01101	A << B	Битовый сдвиг A влево на B разрядов
01110	A >> B	Битовый сдвиг A вправо на B разрядов
01111	A == B	Результат равен 1, если A равно B, иначе 0
10000	A != B	Результат равен 1, если A не равно B, иначе 0
10001	A < B	Результат равен 1, если A меньше B, иначе 0
10010	A > B	Результат равен 1, если A больше B, иначе 0
10011	A <= B	Результат равен 1, если A меньше или равен B, иначе 0
10100	A >= B	Результат равен 1, если A больше или равен B, иначе 0

Операции АЛУ выполняются по модулю 2^{32} , что эквивалентно взятию 32 младших бит результата. Проверка деления на ноль не реализована и является обязанностью программиста.

1.9. Цикл исполнения инструкции

Каждая инструкция в процессоре выполняется за четыре такта, при этом перекрытие стадий исполнения между инструкциями отсутствует. Выполнение инструкции строго последовательно и соответствует диаграмме, приведённой на рисунке ниже.

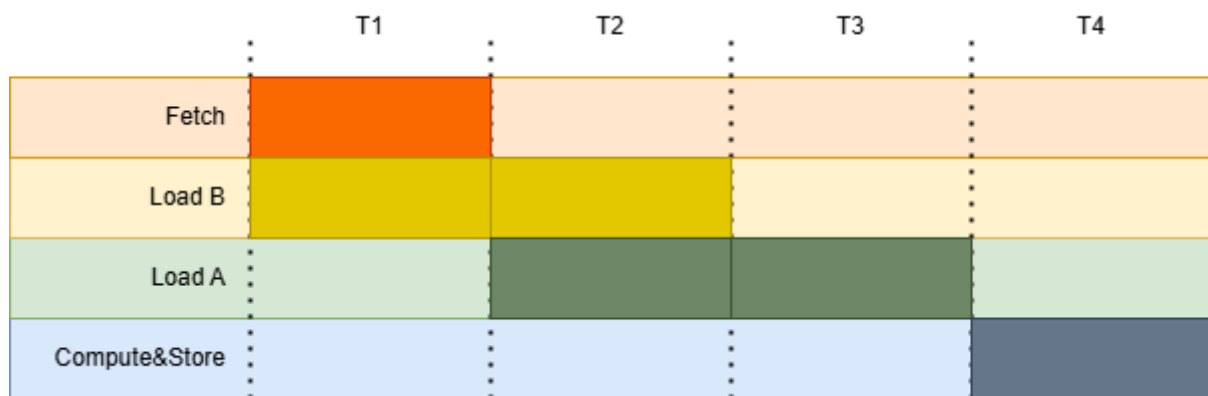


Рисунок 2. Диаграмма исполнения инструкции

На первом такте происходит декодирование инструкции, расположенной по адресу, хранящемуся в регистре PC. После выборки инструкции значение PC инкрементируется для подготовки к чтению следующей инструкции.

Формируются и фиксируются актуальные значения управляющих сигналов. Кроме того, вычисляется адрес для чтения операнда В и устанавливается на адресную шину.

Если операнд В должен быть считан из памяти, на шину выставляется соответствующий адрес памяти; если же операнд В берётся из регистра, на адресную шину устанавливается нулевое значение. Если операнд читается со стека, то происходит уменьшение значения регистра SP на единицу.

Все изменения состояния регистров и управляющих сигналов фиксируются по возрастающему фронту тактового сигнала.

На втором такте операнд В становится доступным для чтения и записывается в предназначенный для него внутренний регистр. Значение операнда В может поступать:

- из памяти по адресу, вычисленному на предыдущем такте;
- из одного из регистров процессора.

Параллельно с этим вычисляется адрес для чтения операнда А, который устанавливается на адресную шину. Процедура вычисления адреса аналогична предыдущему такту, однако в качестве адреса загрузки операнда А допускается использование значения операнда В.

На третьем такте операнд А становится доступным для чтения и записывается в соответствующий ему внутренний регистр. После завершения этого такта оба операнда полностью подготовлены к выполнению операции.

На последнем такте выполняются вычисления в АЛУ с использованием ранее сохранённых значений операндов А и В. Если инструкция предусматривает сохранение результата, он записывается в целевое место хранения:

- в память данных,
- в стек вычислений (с увеличением значения SP на единицу),
- либо в один из регистров процессора.

Теоретически, использование пересылки могло бы сократить схему обработки до трех тактов, однако данная оптимизация не была реализована. АЛУ может использовать значения операндов только после их явного сохранения во внутренние регистры, что и определяет четырёхтактную структуру исполнения инструкции.

1.10. Описание функциональных блоков

Разработанный процессор разделён на логические блоки в соответствии с выполняемыми ими функциями. Такое разбиение упрощает как понимание архитектуры, так и последующую модификацию отдельных частей системы. Ниже приведены схемы соединения основных блоков процессора и описание их функционального назначения.

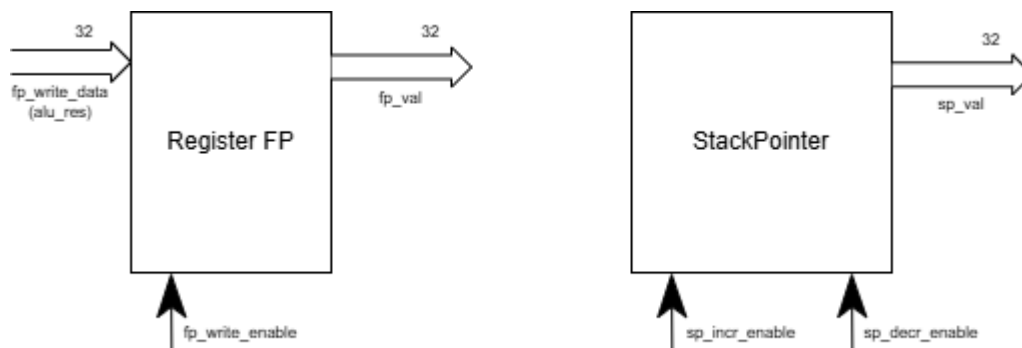


Рисунок 3. Регистры-указатели на вершину стека и фрейм функции

На рисунке показаны регистры указателя вершины стека вычислений и указателя фрейма текущей функции. Регистр фрейма функции используется для адресации локальных переменных, тогда как регистр вершины стека определяет текущую позицию стека вычислений. Назначение и особенности работы данных регистров были подробно рассмотрены в одном из предыдущих разделов, поэтому в данном разделе они приводятся лишь в контексте общей архитектуры процессора.

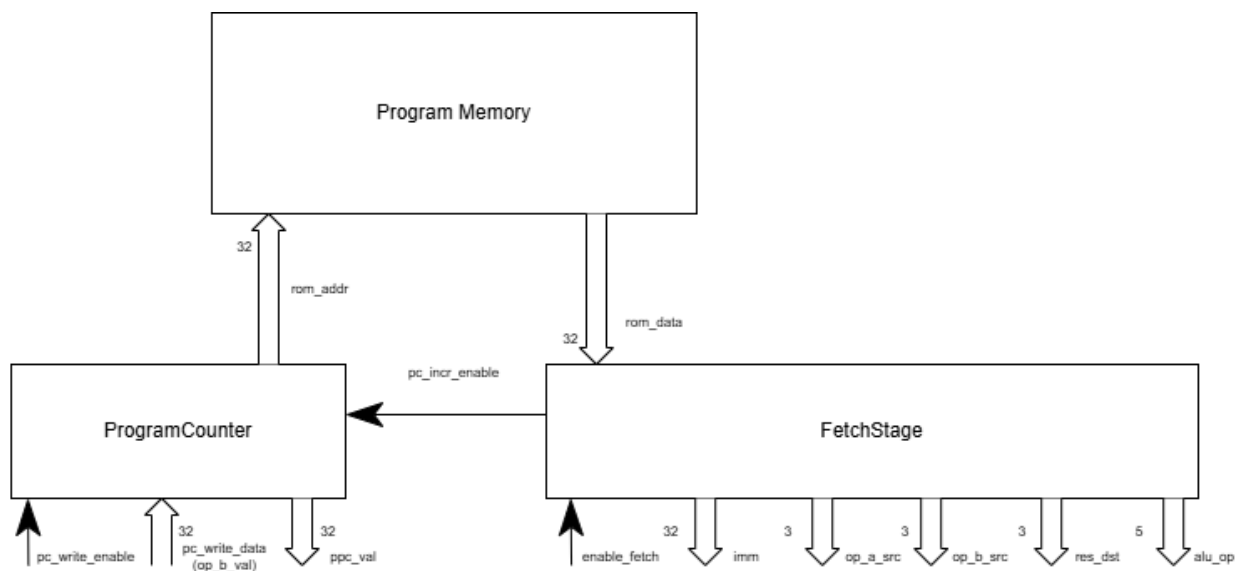


Рисунок 4. Схема соединения блоков выборки и декодирования инструкции

На рисунке представлены блоки, отвечающие за выборку и декодирование инструкций. На адресную шину памяти программ постоянно подаётся значение программного счётчика РС. После изменения значения РС новая инструкция становится доступной для модуля декодирования FetchStage на следующем такте.

Модуль декодирования формирует значения управляющих регистров, которые выводятся на соответствующие шины: `imm`, `op_a_src`, `op_b_src`, `res_dst`, `alu_op`.

Декодирование инструкции и обновление регистров выполняется только при активном сигнале `enable_fetch`.

Для инструкций перехода предусмотрена возможность изменения значения программного счётчика. В этом случае новое значение PC подаётся на шину `pc_write_data`, а его запись производится при активном сигнале `pc_write_enable`.

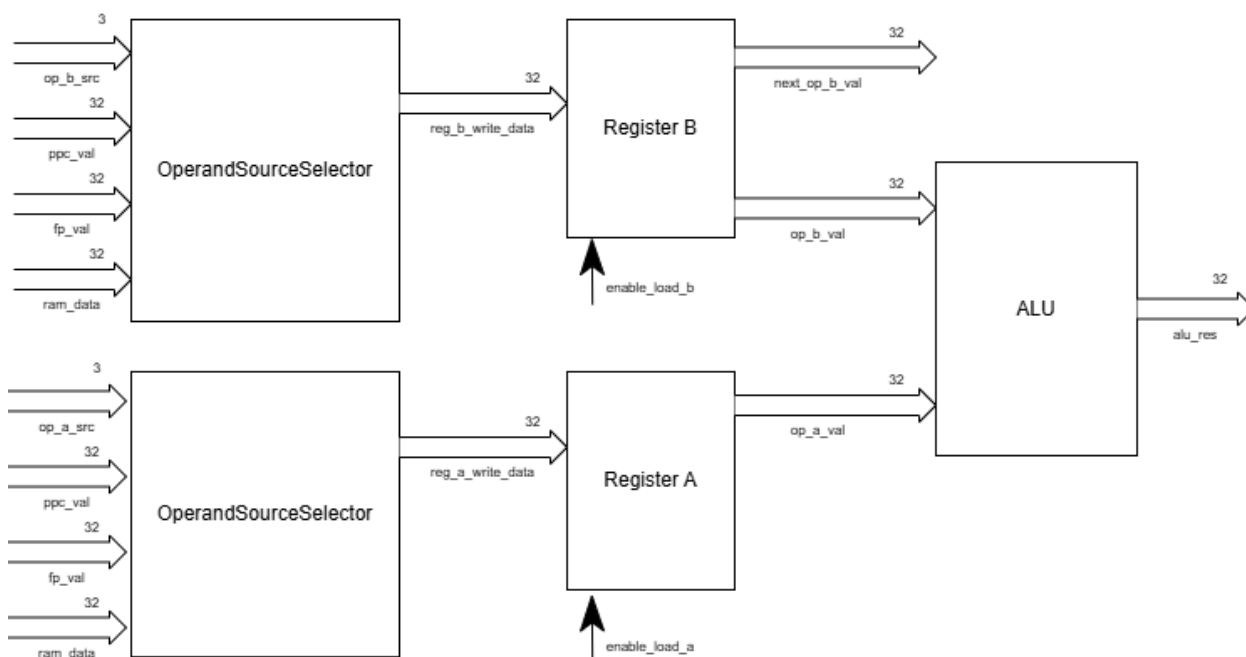


Рисунок 5. Схема соединения блоков загрузки операндов и АЛУ

Загрузка значений операндов в регистры В и А выполняется при активных сигналах `enable_load_b` и `enable_load_a` соответственно. Источники значений для записи выбираются на основе `op_b_src` и `op_a_src`.

Сохранённые в регистрах значения операндов подаются на входы арифметико-логического устройства. Результат работы АЛУ выводится на шины записи регистра FP и памяти данных.

Регистр операнда В имеет дополнительный выход `next_op_b_val`, позволяющий получить вычисленное значение операнда до его фиксации по фронту тактового сигнала. Это используется при формировании адресов и вспомогательных вычислениях, не нарушая общей синхронной модели работы процессора.

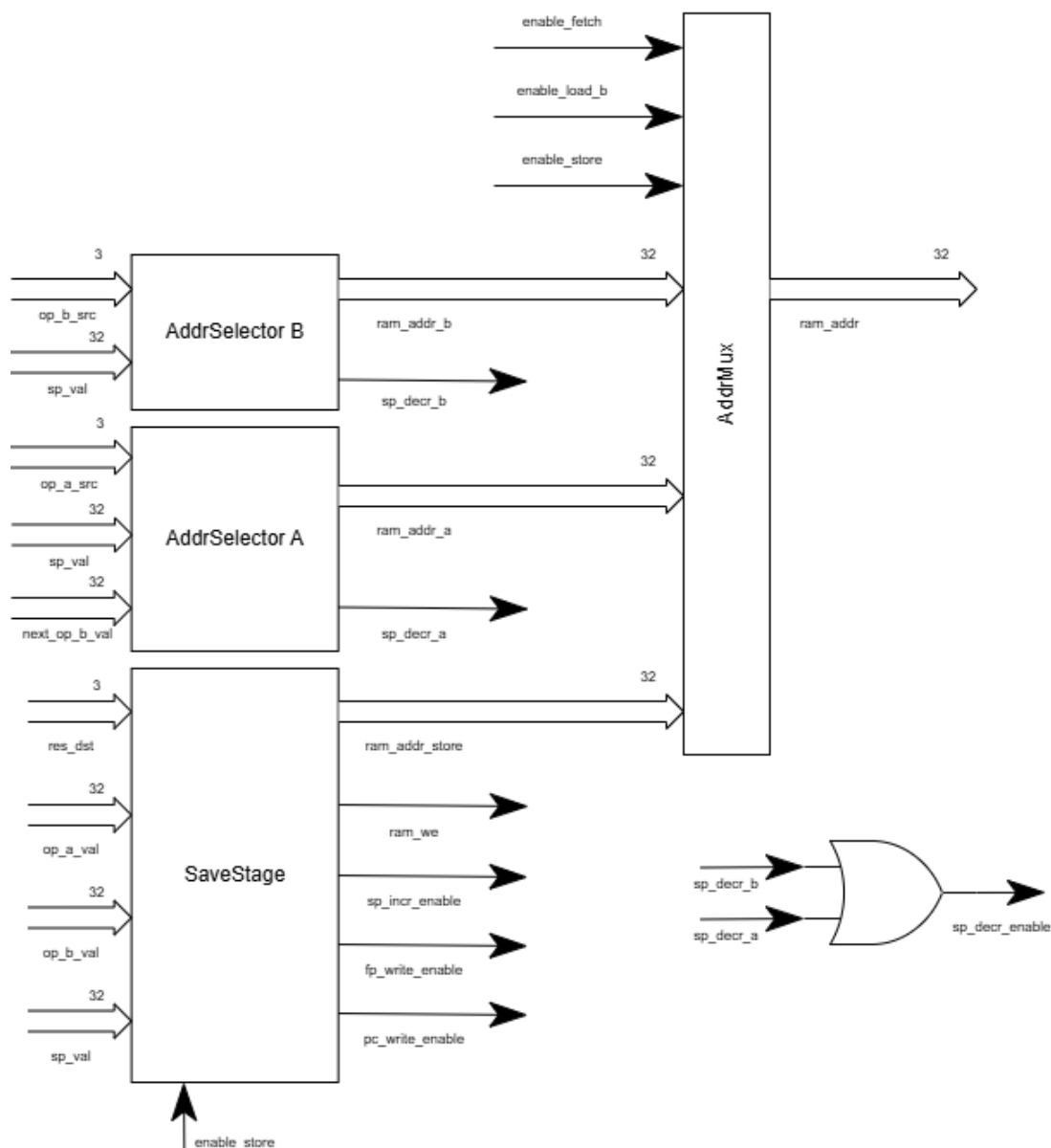


Рисунок 6. Схема соединения блоков вычисления адресов загрузки и сохранения

Адреса загрузки операндов формируются блоками AddrSelector B и AddrSelector A. Эти блоки вычисляют значения, выставляемые на адресную шину памяти данных, а также формируют сигналы управления, отвечающие за уменьшение значения регистра SP при чтении данных со стека.

Адрес сохранения результата, а также соответствующие сигналы разрешения записи, формируются блоком SaveStage. Формирование сигналов происходит только при активном сигнале `enable_store`; в противном случае на выходах блока устанавливаются нулевые значения.

Выбор конкретного источника адреса для памяти данных осуществляется шинным мультиплексором AddrMux, который управляется в зависимости от текущего такта выполнения инструкции.

Важную роль в управлении процессом исполнения играет не отображенный на схеме четырёхбитный циклический сдвиговый регистр. Он формирует номер текущего такта выполнения инструкции в формате one-hot (активен только один бит) и используется для генерации управляющих сигналов `enable_fetch`, `enable_load_b`, `enable_load_a` и `enable_store`.

3. Реализация чтения и записи программ через UART

Базовый способ записи данных в память программ – специальный файл в hex или mif формате, который описывает размещение данных по адресам памяти. Этот способ хорошо работает в случае запуска симуляции, так как файл читается непосредственно при запуске ModelSim. При синтезе проекта под ПЛИС файл читается один раз и внедряется в файл битового потока. После изменения программы требуется выполнить синтез проекта заново, что занимает значительное время, повышая цену ошибки.

Следует внедрить в проект некоторый способ загрузки программы сразу в память работающего устройства. Terasic DE1-SoC включает встроенный преобразователь RS-232 в USB, что упрощает подключение к компьютеру для обмена данными. Поэтому для обмена информацией между устройством и компьютером используется протокол UART.

3.1. Протокол UART

Есть два варианта организации передачи данных между устройствами: параллельный, с использованием определенного количества линий связи, или последовательный, с передачей битов по одному проводнику. UART – это асинхронный протокол, передающий данные в непрерывном потоке сигналов, протекающих по одному проводу в одну сторону. Для организации двусторонней передачи требуется две линии связи (не считая общего провода).

Приемник и передатчик должны быть синхронизированы по скорости (baudrate), ширине передаваемого слова, количеству битов окончания, наличию и виду бита четности.

На рисунке 7 представлена диаграмма передачи, соответствующая ширине слова 8 бит, отключенному биту четности и одному биту окончания.



Рисунок 7. Диаграмма работы UART

Незанятая линия удерживается передатчиком на активном уровне. Это дает дополнительную защиту от помех. Начало передачи обозначается переходом линии на низкий уровень на время равное одному биту. Далее в порядке least significant bit first передается байт сообщения. Передача байта заканчивается стоп-битом (или стоп-битами).

3.2. Использование протокола

В Altera Cyclone V отсутствует аппаратный (hard) блок UART, поэтому применяется программная (soft) реализация – IP-ядро Avalon UART RS-232. Оно работает по протоколу Valid-Ready.

Приём данных:

- IP-ядро выставляет сигнал valid, когда байт доступен для чтения;
- внешнее устройство отвечает сигналом ready, когда готово принять данные;
- при одновременном активном состоянии обоих сигналов байт передаётся, а на следующем такте может быть выдан следующий.

При передаче данных роли меняются: IP-ядро управляет сигналом ready, а устройство – valid.

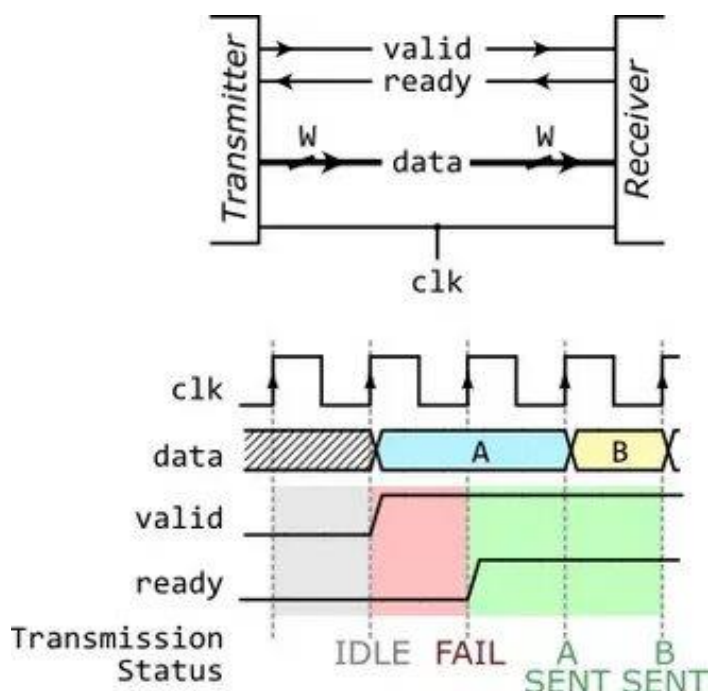


Рисунок 8. Протокол рукопожатия Valid/Ready

Приемопередатчик тактируется частотой в 50МГц, что позволяет ему работать на значительных скоростях. Комфортной скоростью для передачи программного кода небольших размеров является 115200 бод, именно она используется для обмена.

Процессор непрерывно использует память программ, поэтому для безопасного внешнего доступа в качестве памяти программ используется двухпортовое ОЗУ. Порт А используется процессором для чтения инструкций. Порт В управляется UART модулем и используется для чтения/записи значений в ячейки.

Примечание: в Cyclone V все блоки памяти по умолчанию двухпортовые, но один из портов может быть отключён или ограничен (например, только для чтения). Двухпортовость памяти означает существование двух комплектов шин для работы, возможно работающих на разной частоте.

Итак, к одному из портов памяти программ подключен процессор, читающий инструкции одну за другой. К другому порту подключен модуль для изменения содержимого памяти. Этот модуль должен декодировать последовательные байты, приходящие с UART, и исполнять соответствующие команды.

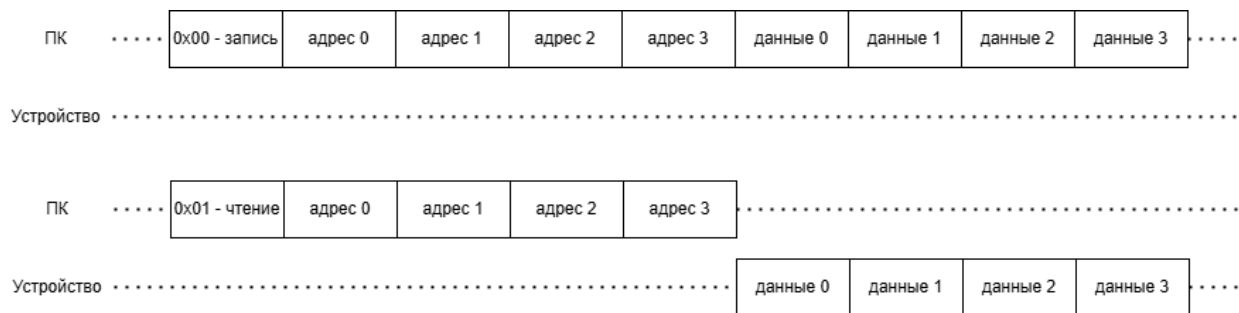


Рисунок 9. Протокол работы с памятью программ

Каждое сообщение состоит из 9 байт. Первым передается байт команды:

- 0x00 – запись в ячейку памяти;
- 0x01 – чтение из ячейки.

Четыре последующих байта задают адрес ячейки в порядке big-endian. На данный момент для адресации 4096 ячеек памяти используется только 12 бит адреса, но формат предусматривает расширение адресного пространства.

Последние четыре байта (также big-endian) передаются либо с устройства на ПК, либо с ПК на устройство в зависимости от выбранной команды.

Декодирование сообщений и управление блоками устройства осуществляется с помощью конечного автомата. Детектирование и сигнализация об ошибках отсутствует, поэтому единственным способом верификации загрузки является повторное считывание содержимого памяти и проверка на совпадение.

На стороне ПК используется скрипт, разработанный на языке программирования Python. Этот скрипт отправляет на устройство последовательность команд, копирующую в память программ заданный массив 32-битных чисел, представляющих инструкции. Копирование начинается с адреса 0 и продолжается, пока в массиве есть нескопированные элементы. После выполнения записи скрипт считывает содержимое памяти и выводит на экран количество неверно записанных инструкций.

После успешной записи программы состояние процессора должно быть сброшено, чтобы выполнение программы началось со стартового адреса. Сброс выполняется путем нажатия на кнопку, размещенную на отладочной плате.

4. Реализация компилятора

Первые тестовые программы для разработанного процессора создавались с использованием ассемблероподобного транслятора, который сопоставлял текстовые имена инструкций их числовым кодам. Такой подход требовал ручного управления размещением переменных в памяти, а также самостоятельного вычисления адресов переходов. Поскольку процессор поддерживает переходы только по абсолютным адресам, вставка нового кода в начало или середину уже существующей программы приводила к необходимости перерасчёта и переписывания всех инструкций перехода. Это существенно увеличивало трудоёмкость разработки и повышало вероятность ошибок.

Разработанный процессор использует собственную систему команд и поддерживает операции только над данными типа `int32`. В связи с этим портирование одного из существующих компиляторов под данную архитектуру потребовало бы несоразмерных затрат времени по сравнению с разработкой самого процессора. Поэтому было принято решение реализовать простой транслятор для высокоуровневого языка, обеспечивающий базовый набор средств, необходимых для удобного написания и отладки тестовых программ.

Для упрощения реализации транслятора используется инструмент ANTLR (ANother Tool for Language Recognition), предоставляющий генератор нисходящих синтаксических анализаторов для формальных языков. Для генерации анализатора требуется задать корректную грамматику в форме РБНФ и выбрать язык программирования, на котором будет написан сгенерированный код анализатора. В качестве целевого языка был выбран C++, а описание грамматики приводится в следующем разделе.

4.1. Описание языка программирования

Описание следует начать с базовых единиц синтаксиса – констант и идентификаторов.

Константы – целочисленные десятичные значения:

NUMBER : [0 – 9] +

Константа задается одним или несколькими десятичными разрядами, при этом допускаются ведущие нули. Обработка отрицательных констант происходит путем применения к неотрицательным константам операции изменения знака. Например, число ‘-001230044’ будет обработано за две инструкции. Сначала на стек вычислений будет загружено значение 1230044, а потом к ней будет применена операция ‘-’.

Идентификаторы используются при задании имен переменных и подпрограмм:

IDENT : [a – zA – Z_][a – zA – Z0 – 9_]*

Они могут состоять из подчеркиваний ‘_’, строчных и заглавных латинских букв и цифр. Цифра не может быть первым символом идентификатора.

Программист может объявлять переменные с произвольными именами. При объявлении переменная обязательно должна быть инициализирована значением некоторого выражения. Дублирование кода можно избежать путем создания процедур.

Таблица 5. Синтаксические конструкции языка

Конструкция	Грамматика	Пример
-------------	------------	--------

Объявление переменной	var_decl_stmt : VAR IDENT ASSIGN expr SEMICOLON;	var foo = 123;
Объявление подпрограммы	func_decl : FUNC IDENT LBRA stmts RBRA	func bar { print(2); }

Файл с кодом должен состоять из:

- объявлений глобальных переменных;
- объявлений подпрограмм.

Тело подпрограммы – это некоторая последовательность операторов, поддерживаемых языком:

Таблица 6. Синтаксические конструкции языка

Конструкция	Грамматика	Пример
Оператор присваивания	assign_stmt : IDENT ASSIGN expr SEMICOLON	foo = 2 * foo;
Оператор вывода	print_stmt : PRINT LPAR expr RPAR SEMICOLON	print(foo % 2);
Условный оператор	if_stmt : IF LPAR expr RPAR stmt (ELSE stmt)?	if (foo % 2) { ... } или if (foo % 2) { ... } else { ... }
Цикл	while_stmt : WHILE LPAR expr RPAR stmt	while (foo > 0) { ... }
Вызов подпрограммы	call_stmt : CALL IDENT SEMICOLON	call bar;

В грамматике используется ключевое слово func, но называть подпрограммы функциями было бы некорректно. Не поддерживается передача параметров и возврат результата. Отсутствующий функционал может быть заменен глобальными переменными. Процедуры могут вызывать друг друга, что позволяет реализовывать рекурсивные алгоритмы.

Для записи арифметических и побитовых выражений поддерживаются все операции, которые были упомянуты среди операций АЛУ. При определении последовательности действий компилятор опирается на расстановку скобок и приоритеты операций. Приоритеты операций соответствуют приоритетам в языке C++, а именно:

Таблица 7. Приоритеты операторов

Приоритет	Оператор	Описание
1	-A, ~A	Префиксные унарные операторы
2	A B	Побитовое ИЛИ
3	A ^ B	Побитовое исключающее ИЛИ
4	A & B	Побитовое И
5	A == B, A != B	Операторы проверки равенства

6	$A < B, A \leq B, A > B, A \geq B$	Операторы проверки неравенств
7	$A \ll B, A \gg B$	Операторы битового сдвига
8	$A + B, A - B$	Операторы суммирования
9	$A * B, A / B, A \% B$	Операторы умножения, деления

Сверху вниз операторы указаны в порядке убывания приоритета. Все операции выполняются слева направо.

4.2. Работа с памятью

Переменные в языке могут объявляться как на верхнем уровне программы, так и внутри отдельных блоков кода. В первом случае переменные доступны из любой части программы и называются глобальными. Во втором случае переменные объявляются внутри блока и доступны только в пределах этого блока и всех вложенных в него блоков, что соответствует понятию локальных переменных.

Локальные переменные размещаются в стековом фрейме функции. Перед началом выполнения подпрограммы компилятор изменяет значение указателя фрейма FP на требуемое количество слов, тем самым резервируя память под локальные данные. После завершения выполнения подпрограммы значение FP изменяется обратно, освобождая ранее выделенную область памяти.

На этапе генерации кода подпрограммы компилятору заранее неизвестно, сколько локальных переменных будет создано во время её выполнения. По этой причине инструкции изменения значения указателя фрейма формируются после завершения генерации кода подпрограммы. Для этого компилятор поддерживает стек позиций инструкций, отвечающих за изменение FP , которые впоследствии корректируются с учётом фактического числа локальных переменных.

Глобальные переменные размещаются, начиная с 200-й ячейки памяти данных. Адреса с меньшими номерами зарезервированы под стек вычислений и не используются для хранения глобальных данных.

Для обеспечения корректной области видимости переменных реализован механизм контекстов видимости. Компилятор поддерживает стек контекстов, каждый из которых соответствует отдельному блоку кода и содержит информацию о переменных, объявленных в этом блоке, а также о присвоенных им адресах в памяти. Границы контекстов совпадают с границами блоков, задаваемых фигурными скобками в исходном коде. Глобальные переменные хранятся в контексте глобального уровня, который является первым элементом данного стека.

4.3. Генерация кода

Генерация машинного кода выполняется в процессе обхода синтаксического дерева программы. Для каждой грамматической конструкции, описанной в грамматике, ANTLR автоматически формирует соответствующий метод обработки в классе `BaseVisitor`. Переопределяя эти методы в классе-наследнике, можно управлять порядком обхода дерева и задавать действия, выполняемые при обработке каждого узла.

Генерация инструкций, реализующих операции работы с памятью и арифметико-логические вычисления, в условиях стековой модели вычислений не представляет существенной сложности. Основным требованием является соблюдение корректного порядка размещения операндов на стеке: перед выполнением бинарной операции на вершине стека вычислений должен находиться правый операнд, а непосредственно под ним – левый.

Сложнее дела обстоят с генерацией инструкций перехода, ведь они требуют указания конкретного адреса, который до полной генерации кода остается неопределенным. Для решения этой проблемы в генераторе кода реализуется механизм меток. Для каждой ячейки, где должен быть подставлен адрес перехода, в специализированную таблицу по имени метки записывается адрес места вставки.

Когда адрес перехода становится известным, с меткой через таблицу связывается значение, которое должно быть установлено во всех местах, где оно требуется. Процесс установки значений происходит отдельным проходом после генерации кода.

Ниже представлен пример кода, выполняющего вычисление факториала числа, и результат его компиляции:

Программа вычисления факториала		
<pre> var n = 5; var result = 1; func factorial { var i = 1; while (i <= n) { result = result * i; i = i + 1; } } func main { call factorial; print(result); // Expected output: 120 } </pre>		
	0 0x00000400	1024
	1 0x800000b9	A : stack, imm => fp imm=0
	2 0x00000005	5
	3 0x000000c8	200
	4 0x800000b5	A : stack, stack => [B] imm=0
	5 0x00000001	1
	6 0x000000c9	201
	7 0x800000b5	A : stack, stack => [B] imm=0
	8 0x00000024	36
	9 0x800000bb	A : stack, imm => pc imm=0
L0:		
	10 0x0000000a	10
	11 0x800000bb	A : stack, imm => pc imm=0
factorial:		

	12 0x800000be	A : ppc, imm => stack imm=0
	13 0x8000022a	ADD : fp, imm => fp imm=1
	14 0x00000001	1
	15 0x800000b7	A : stack, fp + imm => [B] imm=0
L1:		
	16 0x800000b6	A : [B], fp + imm => stack imm=0
	17 0x000000c8	200
	18 0x800000b4	A : [B], stack => stack imm=0
	19 0x80000130	LE : stack, stack => stack imm=0
	20 0x00000022	34
	21 0x800000bc	A : stack, stack => a == 0 ? pc : nop imm=0
	22 0x000000c9	201
	23 0x800000b4	A : [B], stack => stack imm=0
	24 0x800000b6	A : [B], fp + imm => stack imm=0
	25 0x80000040	MUL : stack, stack => stack imm=0
	26 0x000000c9	201
	27 0x800000b5	A : stack, stack => [B] imm=0
	28 0x800000b6	A : [B], fp + imm => stack imm=0
	29 0x00000001	1
	30 0x80000020	ADD : stack, stack => stack imm=0
	31 0x800000b7	A : stack, fp + imm => [B] imm=0
	32 0x00000010	16
	33 0x800000bb	A : stack, imm => pc imm=0
L2:		
	34 0xfffffe2a	ADD : fp, imm => fp imm=-1
	35 0x800000bb	A : stack, imm => pc imm=0
main:		
	36 0x800000be	A : ppc, imm => stack imm=0
	37 0x8000002a	ADD : fp, imm => fp imm=0
	38 0x0000000c	12
	39 0x800000bb	A : stack, imm => pc imm=0
	40 0x000000c9	201
	41 0x800000b4	A : [B], stack => stack imm=0
	42 0x40000000	1073741824
	43 0x800000b5	A : stack, stack => [B] imm=0
	44 0x8000002a	ADD : fp, imm => fp imm=0
	45 0x800000bb	A : stack, imm => pc imm=0

В левой части отображения сгенерированного кода представлены метки, размещённые непосредственно над инструкциями, на которые они ссылаются. Каждая строка соответствует одной инструкции. В начале строки указывается номер ячейки памяти программ, в которую должна быть загружена инструкция, затем приводится её шестнадцатеричное представление и текстовая расшифровка.

Если в расшифровке указано только числовое значение, то данная инструкция соответствует загрузке константы на стек. В остальных случаях сначала указывается выполняемая операция АЛУ, затем режим адресации, и в конце – значение константы, встроенной в инструкцию.

Помимо кода, непосредственно написанного программистом, транслятор автоматически добавляет служебный код, выполняемый до вызова процедуры `main`. В начале исполнения программы указатель фрейма функции устанавливается на адрес 1024. Стек фреймов функций при этом растёт в направлении увеличения адресов памяти. Данное решение не было описано в разделе, посвящённом аппаратной части, поскольку оно полностью относится к уровню реализации компилятора.

Кроме того, до передачи управления в `main` выполняется инициализация глобальных переменных. В инструкциях 8–9 осуществляется безусловный переход к началу процедуры `main`.

Тело подпрограммы начинается с сохранения адреса возврата в стек вычислений и увеличения значения `FP`, то есть с выделения памяти под локальные переменные. В конце подпрограммы значение `FP` возвращается к исходному, после чего происходит возврат из подпрограммы путём извлечения адреса возврата из стека вычислений и его записи в регистр `PC`. В случае выхода из процедуры `main` данная операция соответствует инструкции 45.

Помимо меток, соответствующих именам подпрограмм, транслятором были сгенерированы вспомогательные метки `L0`, `L1` и `L2`. Метка `L0` используется для возврата из процедуры `main` и указывает на бесконечный цикл, который генерируется транслятором для предотвращения выхода процессора за пределы существующего программного кода. Метки `L1` и `L2` обозначают соответственно начало и конец цикла `while`, расположенного внутри подпрограммы `factorial`.

Таким образом, был реализован минималистичный компилятор для целевой платформы, позволяющий разработчику абстрагироваться от работы с машинным кодом и ручного управления адресами памяти. При этом компилятор не выполняет проверок корректности адресации и использования ресурсов. В связи с этим программисту рекомендуется соблюдать определённые ограничения: избегать чрезмерно глубокой вложенности выражений, глубокой рекурсии и одновременного использования большого количества переменных.

4.4. Примеры программ

Сумма чисел от 1 до N
<pre> var N = 10; var sum = 0; func calcSum { var i = 1; while (i <= N) { sum = sum + i; i = i + 1; } } func main { call calcSum; print(sum); // Expected output: 55 } </pre>

Проверка четности числа

```
var num = 13;
var isEven = 0;

func checkEven {
    if (num % 2 == 0) {
        isEven = 1;
    } else {
        isEven = 0;
    }
}

func main {
    call checkEven;
    print(isEven); // Expected output: 0 (нечётное)

    num = 8;
    call checkEven;
    print(isEven); // Expected output: 1 (чётное)
}
```

Табуляция функции $y = x^2 - 5x + 6$

```
var x = 0;
var y = 0;

func computeY {
    y = (x * x) - (5 * x) + 6;
}

func main {
    while (x <= 10) {
        call computeY;
        print(x);
        print(y);
        x = x + 1;
    }
}
```

Заключение

В ходе выполнения работы был реализован учебный процессор со стековой архитектурой на ПЛИС Altera Cyclone V. Построена полная аппаратная модель вычислительной машины: разработаны регистры, подсистема памяти, система команд, декодер, АЛУ и механизм исполнения инструкций. Создан загрузчик программ через UART, позволяющий изменять память без повторного синтеза. Реализован упрощённый компилятор на основе ANTLR, поддерживающий переменные, условные операторы, циклы и подпрограммы.

Полученная система позволяет изучать принципы работы процессоров на реальном оборудовании и служит основой для дальнейших расширений: внедрения конвейеризации, оптимизации команд, поддержки прерываний, расширения языка программирования. Проект демонстрирует полный цикл проектирования аппаратного и программного обеспечения и подтверждает работоспособность предложенной архитектуры как в симуляции, так и на реальной ПЛИС.

Список литературы

1. Харрис С., Харрис Д. *Цифровая схемотехника и архитектура компьютера: RISC-V* / пер. с англ. В. С. Яценкова, А. Ю. Романова; под ред. А. Ю. Романова. – М.: ДМК Пресс, 2021. – 810 с.: ил. – ISBN 978-5-97060-961-3.
2. Романова И. И., Американов А. А., Романов А. Ю. *Проектирование FPGA в Quartus* / под ред. А. Ю. Романова. – М.: ДМК Пресс, 2025. – 284 с.: ил. – ISBN 978-5-93700-415-4.
3. Chip Design School [Электронный ресурс] // YADRO Engineer. – URL: <https://engineer.yadro.com/chip-design-school/> (Дата обращения: 18.10.2025).

Приложение А. Исходный код

Исходный код проекта FPGALearning — Calc2 [Электронный ресурс] / репозиторий FPGALearning. – GitHub. – URL:
<https://github.com/AlexixRugis/FPGALearning/tree/main/Calc2> (Дата обращения: 11.12.2025).